

Analizador de Sequências de DNA

Desenvolvimento de software em Arquitetura Modular (Linguagem C)
Trabalho Prático Final

Integrantes:

ANDERSON ANTÔNIO JUSTINO DE OLIVEIRA
JOSÉ FABRICIO SUTER GAZOLA
RAQUEL CRISTINA ANDRÉ
RENATA KIYOCO HARA GIL
VINICIUS OTÁVIO CAMPOS LOPES

Orientador:

Profº Alex Marino Gonçalves de Almeida

2º termo ADS Vespertino • Data: [09/06/2026]

DEFINIÇÃO E CONTEXTUALIZAÇÃO

O QUE É O DNA?

Biologicamente, o DNA é a molécula portadora da informação genética, constituída por uma sequência de bases nitrogenadas: **Adenina (A)**, **Timina (T)**, **Citosina (C)** e **Guanina (G)**.

ABORDAGEM COMPUTACIONAL

No sistema, esta sequência é tratada como um **vetor de caracteres (string)**. Esta abstração permite a aplicação de algoritmos clássicos de manipulação de *strings* para resolver problemas de bioinformática, como alinhamento, busca de padrões e análise de similaridade.

MOTIVAÇÃO

A necessidade de ferramentas eficientes para a área de Bioinformática justifica o desenvolvimento deste software, que integra manipulação de dados genéticos com o rigor da linguagem C.



```
char dna[] = "ATGCCGTAA";
```

// String com 4 caracteres possíveis: a base da bioinformática.

Empresas como Illumina, Roche e laboratórios de pesquisa precisam de programadores que saibam manipular sequências.

ARQUITETURA MODULAR

Paradigma Modular

Código segmentado em 6 módulos, facilitando a legibilidade e a manutenção.

Automação

Gerenciamento de *build* simplificado via *Makefile* (comandos *make*, *make run*, *make clean*).

Qualidade

Conformidade com o padrão C99 e compilação rigorosa (flag *-Wall*) para detecção proativa de erros.

Mapeamento de Componentes

Módulo	Responsabilidade
main.c	Menu principal e integração
cadastro	Inclusão, listagem, validação
analise	Contagem, GC, motif
Ordenacao	Algoritmo de ordenação
Arquivo	Salvar / carregar do disco
Comparacao	Distância de Hamming

FUNCIONALIDADES TÉCNICAS DO SISTEMA

Parte I (Básico)

Interface via menu (*do-while* / *switch-case*). Foco na **integridade dos dados**: qualquer entrada que fuja ao alfabeto {A, T, C, G} é rejeitada pela função de validação.

Parte II (Intermediário)

Utilização de *struct* para encapsular os dados da sequência (ID, Nome, DNA, Tamanho). Implementação do *Bubble Sort* para ordenação por comprimento e *strstr* para busca de *motifs*.

Parte III (Avançado)

Persistência de dados em arquivos (*fopen*, *fprintf*) e implementação do algoritmo de **Distância de Hamming**, que quantifica a divergência entre duas sequências, simulando a detecção de mutações genéticas.

Parte I — Básico

DEMONSTRAÇÃO: FUNCIONALIDADES BÁSICAS

ROTEIRO:

1. **Cadastro:** Cadastrar 10 sequências de DNA de animais.
2. **Validação de Entrada:** Tentar cadastrar sequência inválida contendo a letra 'X', resultando na rejeição imediata pelo sistema.
3. **Listagem:** Exibir todas as sequências em formato de tabela.
4. **Análise de Bases:** Contar bases da sequência selecionada, apresentando o resultado.
5. **Cálculo GC:** Calcular GC e apresentar os percentuais da(s) sequência(s) selecionada(s).

Parte II — Intermediário

DEMONSTRAÇÃO: FUNCIONALIDADES INTERMEDIÁRIAS

ROTEIRO:

1. **Refatorar** a estrutura de dados utilizando *structs* para o encapsulamento de informações (ID, Nome, DNA, Tamanho, GC%).
2. **Ordenar** sequências por tamanho através do algoritmo *Bubble Sort*.
3. **Localizar** padrões biológicos (motifs) utilizando a função *strstr*.
4. **Garantir** a integridade dos registros durante a manipulação do vetor de sequências.

Algoritmo escolhido:

Bubble Sort

A escolha pelo **Bubble Sort** foi pautada na simplicidade e na manutenção do código. Como o foco desta fase é a manipulação e organização de estruturas de dados (*structs*), optamos por um algoritmo que permite um acompanhamento didático de cada troca realizada, assegurando a confiabilidade e a integridade das sequências processadas.

Parte III — Avançado

DEMONSTRAÇÃO: FUNCIONALIDADES AVANÇADAS

Persistência em arquivo:

1. **Salvar** e **Carregar** o banco de dados em arquivos de texto (.txt) para garantir a persistência das informações.
2. **Fechar** e **Reabrir** o sistema, assegurando a recuperação íntegra do estado original dos registros após o encerramento da sessão.
3. **Encapsular** as propriedades das sequências (nome, dna, tamanho) durante as operações de escrita e leitura, mantendo a consistência dos dados.

Parte III — Avançado

DEMONSTRAÇÃO: FUNCIONALIDADES AVANÇADAS

Análise comparativa (Distância de Hamming):

1. **Otimizar** o processamento da função `distanciaHamming` via passagem de ponteiros (`Sequencia *s1`, `Sequencia *s2`), minimizando o uso de memória ao evitar cópias desnecessárias.
2. **Comparar** sequências através do cálculo da Distância de Hamming para **identificar diferenças pontuais** (mutações) entre genomas armazenados.

Simulação de Mutação Pontual:

Sequência	DNA
Original (Leao_01)	ATGCGTACGTTAGCGTAC G
Variante	ATGCGTACGTTAGCGTAC A

Análise: Ao comparar as sequências via `distanciaHamming`, o sistema detecta a substituição da última base (**G** → **A**).

Resultado da Execução:

`Hamming(Leao_01, Leao_Mutante) = 1`

Conclusão: O valor 1 valida a capacidade do sistema em **quantificar alterações moleculares** pontuais, simulando com precisão técnica a variabilidade genética.

Plano de testes executado

Parte	Cobertura	Resultado
I — Básico	T01–T11 (11 testes)	11 de 11 OK
II — Intermediário	T12–T16 (5 testes)	05 de 05 OK
III — Avançado	T17–T22 (6 testes)	06 de 06 OK
TOTAL	22 testes	22 de 22 OK

Plano de testes executado

DIAGNÓSTICO E CORREÇÃO DE BUGS

Abaixo, apresentamos a distribuição dos 12 problemas técnicos identificados durante o desenvolvimento e como foram solucionados:

Categoria	Qtd	Descrição Resumida	Forma de Correção
Ponteiros	2	Uso de *, & e passagem de parâmetros	Correção de endereçamento e referência
Bubble Sort	2	Lógica de comparação e troca de elementos	Refinamento da lógica de ordenação
Structs	2	Inicialização e acesso aos campos	Ajuste de acesso aos dados da estrutura
Outros (Lógica/Sintaxe/IO)	6	Ajustes diversos de rotina	Revisões de código e validações
TOTAL	12	Problemas corrigidos	—

Dificuldades

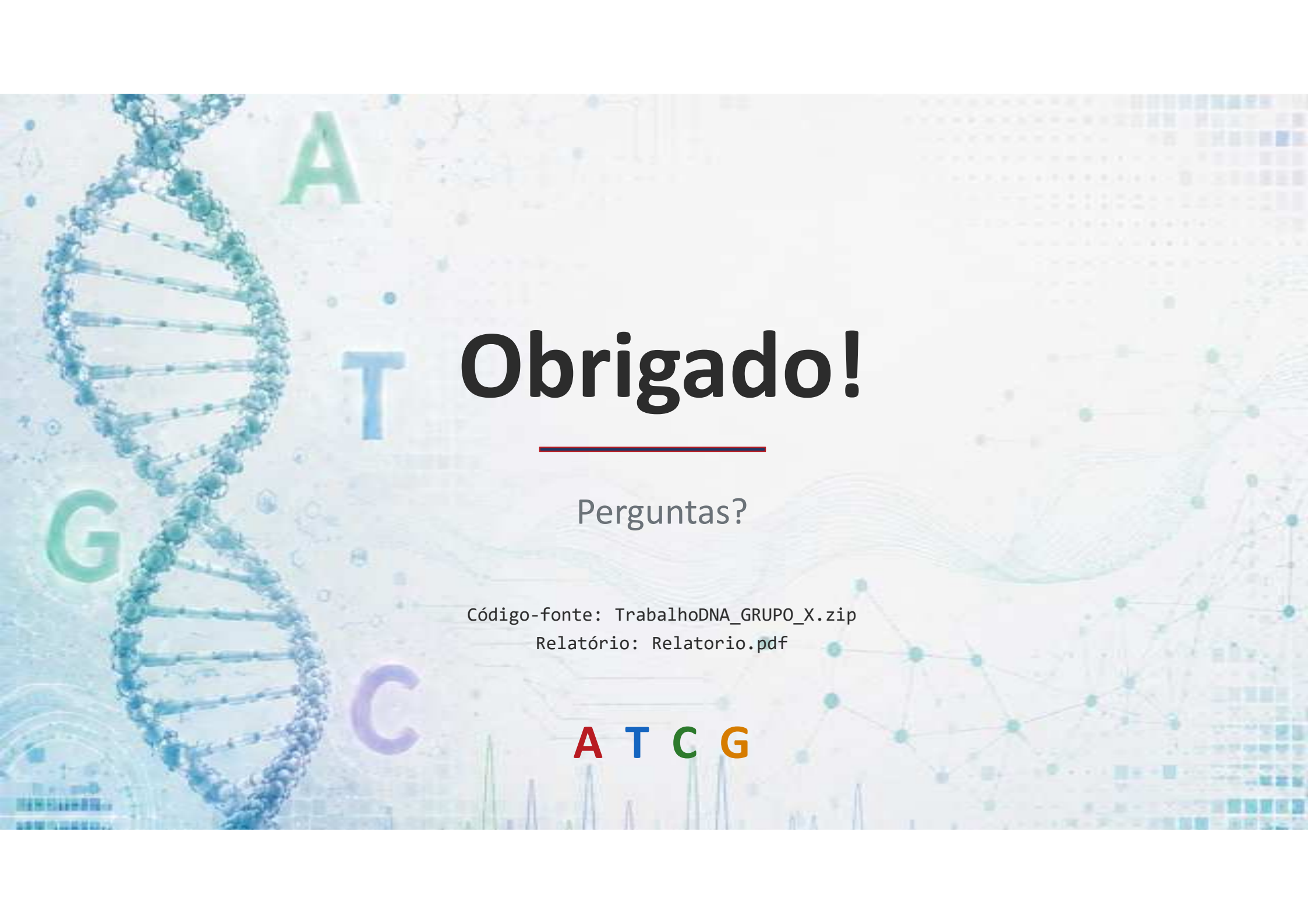
Desafios técnicos superados:

- **Manipulação de ponteiros:** Confusão entre os operadores `*` e `&` e a passagem de parâmetros, corrigida pela prática rigorosa de passagem por referência.
- **Gestão de estruturas (Structs):** Dificuldade inicial na correta inicialização e no acesso aos campos, superada com foco no gerenciamento de memória em structs.
- **Lógica de ordenação:** Desafios na implementação do *Bubble Sort*, especificamente na lógica de comparação e troca de registros.
- **Persistência e fluxo:** Correções necessárias em rotinas de leitura/gravação de arquivos e ajustes nos limites de laços `for`.
- **Coordenação do grupo:** Resolvido com orientações do GitHub, roteiros disponibilizados pelo professor e reuniões curtas.

Aprendizados

Principais aprendizados:

- **Fundamentos de Bioinformática:** A transição da teoria para a prática permitiu compreender como estruturas de dados e algoritmos de ordenação são fundamentais para o processamento eficiente de sequências genéticas.
- **Arquitetura Modular:** A divisão em arquivos .c e .h foi essencial para isolar responsabilidades, facilitar o *debugging* e permitir a escalabilidade do sistema.
- **Superação e Evolução Técnica:** O desafio de dominar conceitos que inicialmente não conhecíamos, como o controle avançado de memória e a manipulação de ponteiros, nos proporcionou uma visão profissional sobre o desenvolvimento em C.
- **Rigor e Qualidade de Código:** O uso sistemático de validação de entradas e flags de compilação (ex: -Wall) foi crucial para assegurar a estabilidade e a correção da aplicação antes da execução.
- **Colaboração Sinérgica:** O trabalho em equipe, mediado pelo GitHub e por reuniões ágeis, foi o diferencial para integrar conhecimentos distintos e transformar desafios técnicos complexos em soluções coletivas robustas.



Obrigado!

Perguntas?

Código-fonte: TrabalhoDNA_GRUPO_X.zip

Relatório: Relatorio.pdf

A T C G